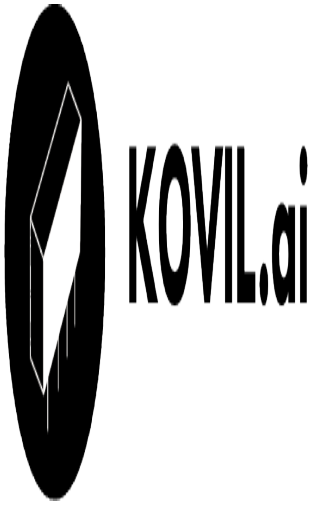

WHITEPAPER · 20 PAGES · FREE DOWNLOAD

GCP AI Agent Architecture Whitepaper

A technical deep dive into the Vertex AI agent stack: Model Garden, Agent Builder vs Reasoning Engine, Vertex AI Search RAG pipelines, BigQuery ML integration patterns, VPC Service Controls configuration, and phased production rollout framework.

Architecture | Vertex AI | Gemini | RAG | BigQuery ML | VPC

Produced by **Kovil AI Engineering Team** · kovil.ai · June 2026

Contents

01 / Overview

The Vertex AI Agent Stack

Google Cloud Vertex AI is not a single product — it is a layered platform of components, each with a specific role in the agent architecture. Understanding which component belongs where, and when to use each, is the core architectural decision that determines build complexity, cost, latency, and maintainability.

Foundation Models **Model Garden**	The model selection layer — 150+ foundation models available as managed endpoints. Includes Gemini 2.0 Flash (speed/cost), Gemini 2.0 Pro (reasoning), Claude 3.5 (complex analysis), Llama 3.3 (open weight, controllable), and Mistral (European data residency). Model selection is the first architecture decision.
Agent Orchestration **Agent Builder / Reasoning Engine**	The agent execution layer — where agent logic, tool calling, grounding, and multi-step reasoning happen. Two options with different trade-offs: Agent Builder (managed, lower engineering effort) and Reasoning Engine (custom, higher control). Detailed comparison in Chapter 3.
Retrieval & Grounding **Vertex AI Search**	The knowledge retrieval layer — indexes your enterprise data and retrieves semantically relevant chunks to ground agent responses. Supports hybrid (keyword + vector) retrieval. The backbone of RAG architectures on GCP.
Structured Intelligence **BigQuery ML**	The structured data reasoning layer — ML models running directly in BigQuery. Integrates with Vertex AI agents for prediction tasks (churn, demand, fraud) without moving data out of the warehouse.
Governance **IAM / VPC Service Controls / Cloud Audit Logs**	The security and compliance layer — controls who can access which AI capabilities, enforces network isolation, and provides audit trails for regulatory requirements.

02 / Foundation Models

Model Garden — Foundation Model Selection

Model Garden is Vertex AI's managed model registry — 150+ foundation models available as API endpoints with enterprise SLAs, VPC isolation, and Vertex AI IAM controls. Selecting the right model for your use case is the most consequential architecture decision in your agent build.

Gemini 2.0 Flash

Best for: high-volume, latency-sensitive agents (FAQ deflection, real-time classification). ~1M token context. Lowest cost in the Gemini family. Fastest inference. Use this for any agent that needs to respond in under 2 seconds.

Gemini 2.0 Pro

Best for: complex reasoning, multi-step planning, long-document analysis. Use when accuracy on complex queries matters more than speed or cost. The recommended model for Reasoning Engine-based agents.

Claude 3.5 Sonnet (via Model Garden)

Best for: nuanced analysis, instruction-following, long-context document processing (200K tokens). Available via Vertex AI Model Garden in supported regions. Enterprise SLA applies. Use when Gemini Pro underperforms on specific reasoning tasks.

Llama 3.3 70B (via Model Garden)

Best for: workloads requiring data sovereignty, IP control, or cost control at scale. Open-weight model — can be fine-tuned on proprietary data. Use when you need model-level customisation or when regulatory requirements prohibit third-party model APIs.

Gemini Embedding Models

Best for: Vertex AI Search grounding, semantic similarity, RAG retrieval. text-embedding-004 is the current recommended embedding model for GCP RAG pipelines. Dimensionality: 768. Supports task-type specification (retrieval, classification, clustering).

03 / Orchestration

Agent Builder vs Reasoning Engine

This is the most common architecture question in Vertex AI agent builds. Agent Builder and Reasoning Engine serve different agent complexity profiles — choosing the wrong one creates technical debt that is expensive to resolve post-launch.

Dimension	Agent Builder	Reasoning Engine
Engineering effort	Low — visual + config-driven	High — Python SDK, custom code
Orchestration control	Managed flows, limited branching	Full control (LangChain, custom)
Tool calling	Built-in data stores, Dialogflow CX	Any REST API, custom Python functions
Memory management	Session-level, managed	Custom — Redis, Firestore, Spanner
Multi-agent patterns	Not supported natively	Full support via orchestration code
Latency (avg)	800ms–2s p95	1.2s–4s p95 (higher due to custom code)
Recommended for	FAQ deflection, conversational service, simple RAG agents	SDR agents, complex ops automation, multi-source data agents
Go-live timeline	**1–2 weeks for first agent**	**2–4 weeks for first agent**

04 / Retrieval

Vertex AI Search RAG Pipeline Architecture

Vertex AI Search is Google Cloud's managed enterprise search service and the recommended retrieval layer for RAG pipelines on GCP. Understanding the pipeline architecture — from document ingestion to grounded generation — is prerequisite to configuring a production agent correctly.

The RAG pipeline stages

1. Data Ingestion

Data sources supported: Cloud Storage (GCS) for unstructured documents (PDF, HTML, DOCX, TXT), BigQuery for structured records, Cloud SQL, websites via web crawl, and custom data via the API. Each source type has different configuration — GCS is simplest, web crawl requires URL pattern configuration, custom API requires schema mapping.

2. Chunking Strategy

Vertex AI Search handles chunking automatically, but the chunking configuration significantly affects retrieval quality. Default: ~500-word chunks with 50-word overlap. For technical documentation: smaller chunks (200–300 words) improve precision. For narrative documents: larger chunks (800–1000 words) improve context preservation. Test both on a sample query set before locking in a strategy.

3. Embedding Generation

Vertex AI Search uses Google's text-embedding-004 model to generate vector representations of each chunk. These embeddings are stored in a managed vector index. You do not need to manage embedding infrastructure — this is handled by Vertex AI Search.

4. Index Configuration

The search index supports three retrieval modes: keyword-only (fast, lower accuracy), vector-only (semantic, medium accuracy), and hybrid (recommended for most production deployments). Hybrid retrieval combines keyword precision with semantic recall, then applies semantic re-ranking to the merged result set.

5. Re-ranking

Vertex AI Search includes a managed re-ranker that scores retrieved chunks by relevance to the query before passing them to Gemini. Re-ranking typically improves grounded response accuracy by 15–25% compared to first-pass retrieval alone. Enable it — it adds minimal latency (<100ms) and significant quality improvement.

6. Grounded Generation

The top-k retrieved chunks (typically k=5–10) are injected into the Gemini prompt as grounding context. The agent is instructed to answer from the grounding context only, with citations. Hallucination rate on well-configured RAG pipelines is typically <3% on in-scope queries.

05 / Data Intelligence

BigQuery ML Integration Patterns

BigQuery ML allows ML models to run directly inside BigQuery — no data movement, no separate ML infrastructure, no ETL pipeline. For Vertex AI agents that need to incorporate structured predictions (demand forecasting, churn scoring, fraud detection), BigQuery ML is the most cost-effective and lowest-latency integration pattern.

Pattern 1: Agent → BigQuery ML → Gemini Explanation

The agent calls a BigQuery ML model via a BigQuery API tool, receives the prediction (e.g. churn_score=0.87), then passes the result to Gemini to generate a plain-language explanation. Use case: sales agent explaining churn risk to a rep, combined with recommended actions.

Pattern 2: Scheduled BQML → GCS → Vertex AI Search

Run BigQuery ML predictions on a schedule (e.g. daily churn scores for all accounts). Store results in GCS as structured documents. Index in Vertex AI Search. Agent retrieves current predictions via RAG. Use case: agents that need to reference predictions without triggering live BQML calls at query time.

Pattern 3: Gemini in BigQuery (BQML Extension)

Use the ML.GENERATE_TEXT and ML.CLASSIFY functions in BigQuery SQL to run Gemini inline with analytics queries. Use case: content moderation at scale, entity extraction from customer feedback at rest, automated classification of BigQuery table contents. Runs as a BigQuery job — not an agent call, but can be triggered by an agent via a BigQuery API tool.

Pattern 4: BQML Forecasting → Vertex AI Dashboard Agent

Train a BigQuery ML ARIMA_PLUS time-series model on historical demand data. Agent queries the model endpoint for current forecasts and surfaces them in conversational format to planners. Use case: supply chain, inventory management, revenue forecasting agents.

06 / Security

VPC Service Controls Configuration

VPC Service Controls (VPC-SC) create a security perimeter around your GCP services that prevents data exfiltration even if IAM credentials are compromised. For regulated industries (financial services, healthcare, government), VPC-SC is not optional — it is the difference between a deployment that can pass a security review and one that cannot.

Perimeter Architecture

A VPC-SC perimeter is defined at the project level and specifies which services are included (Vertex AI, BigQuery, Cloud Storage, Cloud Run, etc.). Requests from outside the perimeter to services inside it are blocked — even with valid IAM credentials. The perimeter is the outer boundary of your enterprise AI security posture.

Access Policies

Access policies define who can cross the perimeter boundary under what conditions. An access level might specify: requests from corporate IP ranges only, or requests using device certificates only. This allows your internal developers and agent service accounts to operate normally while blocking external requests.

Ingress and Egress Rules

Ingress rules allow specific external sources to access services inside the perimeter. Egress rules allow services inside the perimeter to call specific external APIs. For Vertex AI agents that call external REST APIs (via Reasoning Engine tool calls), explicit egress rules must be configured for each external endpoint.

Dry-run Mode

Always configure VPC-SC in dry-run mode before enforcement. Dry-run mode logs violations without blocking — allowing you to identify which services and service accounts need perimeter access before turning on enforcement. Most organisations discover 5–15 unexpected dependencies in dry-run mode.

Common Configuration Mistakes

Not including Cloud Run in the perimeter (agent execution environment). Missing egress rule for Vertex AI Search indexing from GCS. Forgetting to add Secret Manager to the perimeter (agent credentials). Not configuring access levels for CI/CD pipelines that deploy to Cloud Run.

07 / Rollout

Phased Production Rollout Framework

A phased rollout reduces production risk and creates feedback loops before full-scale deployment. Kovil AI uses a four-phase framework for every Vertex AI agent deployment.

Phase 1: Internal Alpha (Days 1–7)

Deploy the agent in a staging environment with no external traffic. Internal team members test against a defined set of 50+ test queries covering success cases, edge cases, and out-of-scope queries. Acceptance criteria: >80% accuracy on in-scope queries, 0% hallucination rate on measurable factual queries, <3 second p95 latency.

> **Gate criteria:** 50 test queries, accuracy baseline, latency baseline, no external users

Phase 2: Limited Beta (Days 8–21)

Deploy to a controlled group of 5–20 internal users or a single customer account. Monitor conversation logs daily. Track resolution rate (did the agent resolve the query without escalation), user satisfaction (thumbs up/down), and escalation patterns. Tune agent instructions and grounding configuration based on failure patterns.

> **Gate criteria:** 5–20 beta users, daily log review, 2-week minimum

Phase 3: Canary Rollout (Days 22–35)

Route 10–20% of production traffic to the agent. Monitor all key metrics in real time via Cloud Monitoring dashboards. Set alerting thresholds: escalation rate >25% triggers review, p95 latency >5 seconds triggers investigation, error rate >2% triggers rollback. Keep the previous manual process running in parallel for 100% of queries during this phase.

> **Gate criteria:** 10–20% traffic, parallel human process, real-time monitoring

Phase 4: Full Production (Day 36+)

Ramp to 100% of target traffic. Establish weekly conversation quality reviews — sampling 50–100 conversations per week and assessing quality against the rubric defined in Phase 1. Plan first model or instruction update for week 6 based on observed failure patterns. Set up automated re-indexing schedule for Vertex AI Search data sources.

> **Gate criteria:** Weekly quality review, monthly optimisation cycle

08 / Checklist

Architecture Decision Checklist

Before writing a single line of agent code, these 12 architecture decisions must be made and documented. Changes to any of these decisions post-build are expensive.

#	Decision	Your Answer
1	**Model selection** Which Gemini model (Flash/Pro) or alternative (Claude, Llama) for the core reasoning task?	
2	**Orchestration layer** Agent Builder or Reasoning Engine? (See Chapter 3 comparison matrix.)	
3	**Grounding strategy** Pure RAG (Vertex AI Search), structured query (BigQuery ML), or hybrid?	
4	**Chunking configuration** Chunk size and overlap for Vertex AI Search index? (Document type dependent.)	
5	**Retrieval mode** Hybrid (recommended), vector-only, or keyword-only for Vertex AI Search?	
6	**Memory architecture** Session-only memory, or persistent memory (Firestore/Redis) across sessions?	
7	**Tool set** Which external APIs and internal data sources will the agent call? Are egress rules configured?	
8	**Human handoff** What triggers escalation to a human? What data is passed at handoff?	
9	**VPC-SC perimeter** Which services are in the perimeter? Are ingress/egress rules configured?	
10	**Region** Which GCP region for all Vertex AI services? Data residency requirements satisfied?	
11	**MLOps pipeline** How will agent instructions and grounding data be updated post-launch? Who owns this?	
12	**Evaluation framework** What query set will be used to measure agent quality? Who reviews conversation logs?	

Need help completing this checklist? Kovil AI runs a 2-week Vertex AI Strategy & Readiness engagement that works through every decision in this checklist and produces a written architecture blueprint. Contact us at kovil.ai.